

Unit Testing: JUnit y Mockito

Qué son JUnit y Mockito

- JUnit: Es el framework de pruebas unitarias estándar en el ecosistema de Java. Permite a los desarrolladores escribir y ejecutar bloques de código automatizados para verificar que métodos o clases individuales funcionen exactamente como se espera.
- Mockito: Es un popular framework de simulación (mocking) para Java. Se utiliza principalmente junto con JUnit para aislar el código que estás probando, creando objetos falsos (mocs) que simulan el comportamiento de componentes externos (como bases de datos, APIs externas o servicios complejos).

Qué problema resuelven

Ambas herramientas abordan el desafío fundamental de garantizar la calidad y fiabilidad del software de forma automatizada y eficiente:

- JUnit resuelve el problema de la verificación manual y repetitiva: En lugar de levantar toda la aplicación o probar pantallas a mano cada vez que se hace un cambio, JUnit permite ejecutar cientos de pruebas en segundos para saber si algo se rompió (regresiones).
- Mockito resuelve el problema del acoplamiento en las pruebas: Muchas veces, una clase depende de una base de datos o de un servicio en la nube que puede no estar disponible o ser lento durante una prueba unitaria. Mockito permite "fingir" esas dependencias para probar únicamente la lógica de la unidad de código en cuestión, de forma rápida y sin efectos secundarios (como modificar datos reales).

¿Qué pasa si no se usan?

Prescindir de estas herramientas en proyectos de desarrollo (especialmente en entornos profesionales o de mediana/gran escala) genera consecuencias directas:

- Pruebas manuales costosas y lentas: Todo el control de calidad recae en humanos probando la aplicación pantalla por pantalla, lo que consume horas de trabajo y es propenso a errores humanos.
- Dificultad para detectar errores a tiempo: Los bugs se descubren tarde en el ciclo de desarrollo (o peor, en producción), lo cual encarece exponencialmente su reparación.
- Código rígido y difícil de mantener: Sin la necesidad de escribir pruebas unitarias, los desarrolladores tienden a escribir código altamente acoplado y difícil de modular.
- Miedo a cambiar el código: Ante la ausencia de una red de seguridad automatizada, cualquier refactorización o mejora se convierte en un riesgo alto, ya que nadie puede asegurar con certeza qué parte del sistema se verá afectada.

```

42 ● @ExtendWith(MockitoExtension.class)
43 class CaptorTest {
44
45     private static final Alumno ANA = new Alumno("A01", "Ana Torres", "ana@academyty.mx");
46
47     @Mock private RepositorioAlumnos repoAlumnos;
48     @Mock private RepositorioCursos repoCursos;
49     @Mock private Notificador notificador;
50
51     @InjectMocks private ServicioInscripcion servicio;
52
53     /** @Captor evita escribir ArgumentCaptor.forClass(...) y ademas conserva los genericos. */
54     @Captor private ArgumentCaptor<Alumno> alumnoCapturado;
55     @Captor private ArgumentCaptor<String> motivoCapturado;
56
57     private Curso java101;
58
59 ● @BeforeEach
60     void curso() {
61         java101 = new Curso("JAVA-101", 2);
62     }

```

La imagen muestra una clase de prueba en Java (usando JUnit y Mockito) orientada a realizar pruebas unitarias con el uso de ArgumentCaptor. A continuación, te detallo los elementos principales que se aprecian en el código:

- **Configuración de Mockito con JUnit:** En la línea 42 se utiliza `@ExtendWith(MockitoExtension.class)` para integrar Mockito con JUnit 5, permitiendo el uso de anotaciones de simulación.
- **Definición de mocks:** Las líneas 47 a 49 declaran objetos simulados (`@Mock`) para repositorios y servicios externos (`RepositorioAlumnos`, `RepositorioCursos` y `Notificador`).
- **Inyección de dependencias:** En la línea 51, la anotación `@InjectMocks` crea una instancia de `ServicioInscripcion` e inyecta automáticamente los mocks declarados previamente dentro de ella.
- **Captura de argumentos:** Las líneas 54 y 55 utilizan la anotación `@ArgumentCaptor` para capturar los argumentos (`Alumno` y `String`) que se pasan a los métodos de los mocks durante la ejecución de las pruebas, facilitando su validación posterior.
- **Inicialización previa:** Las líneas 58 a 62 muestran un método anotado con `@BeforeEach` que inicializa un objeto `Curso` (`java101`) antes de ejecutar cada prueba individual.

```

64  @Test
65  @DisplayName("A quien se le mando la confirmacion, exactamente")
66  void aQuienSeLeMando() {
67      when(repoAlumnos.buscar("A01")).thenReturn(Optional.of(ANA));
68      when(repoCursos.buscar("JAVA-101")).thenReturn(Optional.of(java101));
69
70      servicio.inscribir("A01", "JAVA-101");
71
72      // El captor se coloca DONDE iria el matcher, dentro del verify:
73      verify(notificador).enviarConfirmacion(alumnoCapturado.capture(), any());
74
75      Alumno recibido = alumnoCapturado.getValue();
76      assertAll("el alumno que llego al notificador",
77          () -> assertEquals("A01", recibido.matricula()),
78          () -> assertEquals("Ana Torres", recibido.nombre()),
79          () -> assertTrue(recibido.correo().endsWith("@academyty.mx"),
80              "El correo tiene que ser del dominio de la academia"));
81  }
82

```

La imagen muestra un método de prueba unitaria que valida el comportamiento del envío de confirmaciones. A continuación, se detalla paso a paso lo que hace:

- **Definición de la prueba:** Las líneas 64 y 65 utilizan la anotación `@Test` para indicar que es una prueba unitaria y `@DisplayName` para darle un nombre descriptivo.
- **Configuración de los Mocks (Stubbing):** Las líneas 67 y 68 preparan el comportamiento esperado de los repositorios simulados utilizando `when(...)` y `thenReturn(...)`, devolviendo al alumno "ANA" y el curso "java101" cuando se buscan por sus respectivas claves.
- **Ejecución de la acción:** La línea 70 ejecuta el método principal a probar: `servicio.inscribir("A01", "JAVA-101")`.
- **Verificación y captura:** La línea 73 utiliza `verify(...)` para comprobar que el método `enviarConfirmacion` del notificador fue llamado, utilizando el capturador `alumnoCapturado.capture()` para interceptar el objeto `Alumno` que se pasó como argumento.
- **Aserciones múltiples:** Las líneas 76 a 80 utilizan `assertAll` junto con `assertEquals` y `assertTrue` para validar en detalle los datos del alumno capturado, asegurando que su matrícula sea "A01", su nombre "Ana Torres" y que su correo pertenezca al dominio de la academia.

```

83  /**
84   * El caso donde el captor es imprescindible: el MOTIVO del rechazo lo
85   * escribe el servicio por dentro. No tienes ninguna referencia previa
86   * con la que compararlo -- solo puedes atraparlo al vuelo.
87   */
88  @Test
89  @DisplayName("Con que motivo se rechazo")
90  void conQueMotivo() {
91      java101.inscribir("A08");
92      java101.inscribir("A09"); // lleno
93
94      when(repoAlumnos.buscar("A01")).thenReturn(Optional.of(ANA));
95      when(repoCursos.buscar("JAVA-101")).thenReturn(Optional.of(java101));
96
97      assertThrows(CupoLlenoException.class, () -> servicio.inscribir("A01", "JAVA-101"));
98
99      verify(notificador).enviarRechazo(eq(ANA), eq(java101), motivoCapturado.capture());
100
101      assertEquals("cupo lleno", motivoCapturado.getValue());
102  }
103

```

La imagen muestra un caso donde el uso de ArgumentCaptor se vuelve indispensable para una prueba unitaria. A continuación, se detalla paso a paso lo que hace el código:

- **Comentario explicativo (Líneas 83-87):** Explica el escenario clave donde se necesita un capturador: cuando el motivo del rechazo es generado internamente por el servicio y no tienes una referencia previa con la cual compararlo, por lo que solo puedes atraparlo "al vuelo".
- **Definición de la prueba (Líneas 88-90):** Utiliza la anotación `@Test` y un `@DisplayName("Con que motivo se rechazo")` para documentar la intención del test.
- **Preparación del escenario de cupo lleno (Líneas 91-92):** Inscribe a dos alumnos (A08 y A09) para simular que el curso java101 ha alcanzado su capacidad máxima.
- **Configuración de Mocks (Líneas 94-95):** Prepara el comportamiento de los repositorios para que devuelvan a la alumna ANA y el curso java101 cuando se realice la búsqueda.
- **Lanzamiento de excepción (Línea 97):** Utiliza `assertThrows` para verificar que al intentar inscribir a un alumno adicional (A01) en un curso lleno, el servicio lanza correctamente una excepción de tipo `CupoLlenoException`.
- **Verificación y captura (Línea 99):** Utiliza `verify` para comprobar que se llamó al método `enviarRechazo` del notificador, empleando `motivoCapturado.capture()` para interceptar al vuelo el texto del motivo generado internamente.
- **Aserción final (Línea 101):** Utiliza `assertEquals` para validar que el texto capturado dentro del motivo sea exactamente "cupo lleno".

```
104  /**
105   * getAllValues(): todas las llamadas, en orden.
106   * Util para comprobar que se notifico a los tres alumnos correctos.
107   */
108  @Test
109  @DisplayName("getAllValues: los tres alumnos notificados, en orden")
110  void todosLosCapturados() {
111      Curso amplio = new Curso("JAVA-101", 10);
112      Alumno beto = new Alumno("A02", "Beto Ruiz", "beto@academyty.mx");
113      Alumno carla = new Alumno("A03", "Carla Diaz", "carla@academyty.mx");
114
115      when(repoAlumnos.buscar("A01")).thenReturn(Optional.of(ANA));
116      when(repoAlumnos.buscar("A02")).thenReturn(Optional.of(beto));
117      when(repoAlumnos.buscar("A03")).thenReturn(Optional.of(carla));
118      when(repoCursos.buscar("JAVA-101")).thenReturn(Optional.of(amplio));
119
120      servicio.inscribir("A01", "JAVA-101");
121      servicio.inscribir("A02", "JAVA-101");
122      servicio.inscribir("A03", "JAVA-101");
123
124      ArgumentCaptor<Alumno> todos = ArgumentCaptor.forClass(Alumno.class);
125      verify(notificador, org.mockito.Mockito.times(3))
126          .enviarConfirmacion(todos.capture(), any());
127
128      List<Alumno> capturados = todos.getAllValues();
129      assertEquals(3, capturados.size());
130      assertEquals(List.of("A01", "A02", "A03"),
131          capturados.stream().map(Alumno::matricula).toList());
132  }
133
```

La imagen muestra un test unitario diseñado para comprobar la captura de múltiples llamadas utilizando el método `getAllValues()` de un `ArgumentCaptor`. A continuación se detalla su funcionamiento:

- **Comentario explicativo (Líneas 104-107):** Indica que `getAllValues()` recupera todas las llamadas en orden, lo cual es útil para comprobar que se notificó a los tres alumnos correctos.
- **Definición de la prueba (Líneas 108-110):** Se utiliza la anotación `@Test` junto con `@DisplayName("getAllValues: los tres alumnos notificados, en orden")`.
- **Preparación de datos (Líneas 111-114):** Se instancia un curso amplio (amplio) con capacidad para 10 alumnos y dos nuevos alumnos (beto y carla).
- **Configuración de Mocks (Líneas 115-118):** Se configuran los repositorios para devolver a los alumnos (ANA, beto, carla) y al curso cuando sean buscados.
- **Ejecución de acciones (Líneas 120-122):** Se realizan tres inscripciones consecutivas llamando a `servicio.inscribir` para cada uno de los alumnos en el mismo curso.
- **Verificación múltiple (Líneas 124-126):** Se crea una instancia de `ArgumentCaptor` y se usa `verify(..., times(3))` para asegurar que el método `enviarConfirmacion` fue invocado tres veces capturando los argumentos.
- **Aserciones finales (Líneas 128-132):** Se recupera la lista con `getAllValues()`, se valida que su tamaño sea 3 y se comprueba mediante un flujo (stream) que las matrículas capturadas coincidan exactamente con el orden "A01", "A02" y "A03".

```

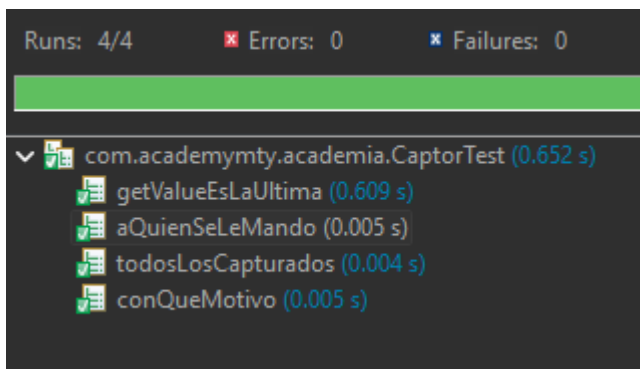
134  /**
135   * AVISO: el captor NO sustituye al matcher.
136   *
137   * Un captor puesto en un verify captura lo que HAYA llegado; no filtra
138   * nada. Si hubo tres llamadas y solo esperabas una, getValue() te
139   * devuelve la ULTIMA sin avisarte. Por eso conviene acompañarlo del
140   * times(n) correspondiente, como en el test de arriba.
141   */
142  @Test
143  @DisplayName("getValue() devuelve la ULTIMA llamada, sin avisar")
144  void getValueEsLaUltima() {
145      Curso amplio = new Curso("JAVA-101", 10);
146      Alumno beto = new Alumno("A02", "Beto Ruiz", "beto@academyty.mx");
147
148      when(repoAlumnos.buscar("A01")).thenReturn(Optional.of(ANA));
149      when(repoAlumnos.buscar("A02")).thenReturn(Optional.of(beto));
150      when(repoCursos.buscar("JAVA-101")).thenReturn(Optional.of(amplio));
151
152      servicio.inscribir("A01", "JAVA-101");
153      servicio.inscribir("A02", "JAVA-101");
154
155      verify(notificador, org.mockito.Mockito.times(2))
156          .enviarConfirmacion(alumnoCapturado.capture(), any());
157
158      assertEquals("A02", alumnoCapturado.getValue().matricula(),
159          "getValue() es la ultima, no la primera");
160      assertEquals("A01", alumnoCapturado.getAllValues().get(0).matricula());
161  }

```

La imagen muestra un test unitario en JUnit y Mockito que demuestra cómo se comporta `getValue()` cuando ocurren múltiples llamadas. A continuación se detalla paso a paso:

- **Comentario explicativo (Líneas 134-141):** Advierte que un capturador dentro de un `verify` no actúa como filtro y captura lo que haya llegado. Explica que si hubo varias llamadas, `getValue()` devuelve directamente la última sin emitir advertencias, por lo que se recomienda usarlo junto con el `times(n)` adecuado.
- **Definición de la prueba (Líneas 142-144):** Se emplea la anotación `@Test` y un `@DisplayName("getValue() devuelve la ULTIMA llamada, sin avisar")`.

- **Preparación de datos (Líneas 145-146):** Se instancia un curso con capacidad amplia y un segundo alumno (beto).
- **Configuración de Mocks (Líneas 148-150):** Se configuran los repositorios para que devuelvan a la alumna ANA, al alumno beto y al curso cuando sean consultados.
- **Ejecución de acciones (Líneas 152-153):** Se realizan dos inscripciones consecutivas llamando a `servicio.inscribir` para A01 y posteriormente para A02.
- **Verificación (Líneas 155-156):** Se usa `verify(..., times(2))` para validar las dos notificaciones de confirmación mientras se capturan los argumentos con `alumnoCapturado.capture()`.
- **Aserciones finales (Líneas 158-160):**
 - Se utiliza `assertEquals` para comprobar que `alumnoCapturado.getValue()` retorna la matrícula del último alumno procesado ("A02").
 - Se valida con `getAllValues().get(0)` que el primer elemento de la lista corresponde al alumno inicial ("A01").



La imagen muestra el panel de ejecución de pruebas de JUnit, indicando el resultado exitoso de la clase `CaptorTest`. A continuación, se detalla su contenido:

- **Resumen general:** En la parte superior se indica que se ejecutaron **4 de 4 pruebas (Runs: 4/4)**, con **0 errores** y **0 fallos** (Errors: 0, Failures: 0), acompañado de una barra de progreso completamente verde que confirma el éxito total.
- **Clase de pruebas:** Se muestra la ruta completa de la suite de pruebas `com.academymty.academia.CaptorTest`, la cual completó su ejecución en un tiempo total de **0.652 segundos**.
- **Casos de prueba individuales:** Se listan los cuatro métodos analizados previamente con su respectivo estado de aprobación (marcado con un icono verde de éxito) y el tiempo que tardó cada uno en ejecutarse:
 - `getValueEsLaUltima` (0.609 s)
 - `aQuienSeLeMando` (0.005 s)
 - `todosLosCapturados` (0.004 s)
 - `conQueMotivo` (0.005 s)